

What the ticket-triage demo actually does, line by line

This is the demo I was shown: one Docker command, one workflow with four nodes, one curl. Every piece of it is explained here in plain words, then the same thing is in this repo as workflow 09 so I can run it and change it.

The terminal, command by command

```
docker rm -f n8n
```

Delete any old container called `n8n`. `-f` means force, even if it is running. A container is a running copy of an image; deleting it does not delete the image or the data volume.

```
docker run -d --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n n8nio/n8n
```

This is the whole `n8n` install. Piece by piece:

- `docker run` : start a new container from an image.
- `-d` : detached, run in the background and give me my terminal back.
- `--name n8n` : call the container `n8n` so I can `docker stop n8n` later.
- `-p 5678:5678` : forward port 5678 on my Mac to port 5678 inside the container. That is why `http://localhost:5678` reaches `n8n`.
- `-v n8n_data:/home/node/.n8n` : mount a named volume. Everything `n8n` saves (its SQLite database with the workflows, credentials, executions) lives in `/home/node/.n8n` inside the container. Without the volume, `docker rm` would erase it all. With it, the data survives.
- `n8nio/n8n` : the image, pulled from Docker Hub the first time.

```
docker stop n8n
docker start n8n
docker ps -a --filter name=n8n
```

Pause, resume, list. `ps -a` shows stopped containers too.

```
docker run --rm -v n8n_data:/d alpine sh -c 'apk add -q sqlite && sqlite3 /d/database.sqlite
"UPDATE ..."'
```

This one failed on his screen (`no such column: "personal"`). He was trying to edit `n8n`'s SQLite database directly to fix workflow ownership after an import, and the shell ate his quotes. Lesson: do not hand-edit `n8n`'s database; import with the `n8n` CLI (`n8n import:workflow`) which sets ownership correctly, which is what `demo/quickstart-docker.sh` does.

The workflow, node by node

Webhook (path `ticket-triage`, method POST, "Respond: using Respond to Webhook node"). Creates the URL `http://localhost:5678/webhook/ticket-triage`. When something POSTs there, the workflow starts with the request body as its input.

OpenAI - Triage (Structured Output). An HTTP Request node that calls the OpenAI API with the ticket text and a JSON schema, so the model must answer in a fixed shape (category, priority, sentiment, summary, key issues, requires_human, suggested_reply) instead of free text. Mine calls Anthropic the same way, with a tool definition instead of a response schema; the idea is identical: you hand the model a schema and it fills it in.

Parse Triage JSON. A Code node. The model's answer arrives wrapped in the API's envelope (choices, message, content, usage). This node digs out the JSON object and puts it under a clean key, `triage`, plus the token counts.

Respond to Webhook. Sends that object back as the HTTP response to whoever called the webhook. Without this node the caller only gets "Workflow was started".

Why the browser said 404

He pasted the webhook URL into Chrome and got:

```
{"code":404,"message":"This webhook is not registered for GET requests. Did you mean to make a POST request?"}
```

A browser address bar always sends a GET. The Webhook node was set to POST. So n8n has nothing registered for GET on that path and says so. That is not a bug; it is the node doing exactly what it was told. To send a POST you need a tool that can: curl, Postman, or another system's webhook step.

The curl, flag by flag

```
curl -X POST http://localhost:5678/webhook/ticket-triage \  
  -H "Content-Type: application/json" \  
  -d '{"ticket": "I was charged twice this month and your app keeps logging me out. Fix this or I am cancelling."}'
```

- `curl` : send an HTTP request from the terminal.
- `-X POST` : use the POST method (default is GET).
- `-H "Content-Type: application/json"` : a header telling the server the body is JSON. Without it n8n would treat the body as a form or plain text and `$json.body.ticket` would be empty.
- `-d '...'` : the body. Single quotes so the shell leaves the double quotes inside alone.
- The trailing `\` at the end of a line means "the command continues on the next line".

What came back (his run, execution #94, 3.18 seconds):

```
{"ok":true,"triage":{"category":"billing","priority":"urgent","sentiment":"angry",  
  "summary":"Customer charged twice and facing app login issues","key_issues":[...],  
  "requires_human":true,"suggested_reply":"I'm sorry to hear about the issues..."},  
  "usage":{"prompt_tokens":322,"completion_tokens":119,"total_tokens":441}}
```

Tokens are the units the model bills by, roughly three quarters of a word each. 322 in, 119 out, 441 total: a fraction of a cent.

What the Executions tab shows

Every run of the workflow, with a timestamp, status, and duration. Click one and the canvas replays it: every node shows its input on the left and its output on the right. That is how you debug: find the node that went red, read what it received, read what it sent. Execution #94 on his screen is the curl above; the green sticky note is him leaving the curl command inside the workflow so the next person knows how to call it.

Same thing in this repo

- `workflows/09-support-ticket-triage.json` : the four nodes above, plus a Config node and a "no ticket, answer 400" branch.
- `demo/quickstart-docker.sh` : his exact `docker run`, then import, publish, the GET that 404s, the POST that works, all printed so you can follow along.
- Needs `ANTHROPIC_API_KEY=...` in `.env.local` (that file is git-ignored; the key never goes in a workflow file).

To swap it to OpenAI: change the URL to `https://api.openai.com/v1/chat/completions`, the header to `Authorization: Bearer ...`, and move the schema into `response_format`. Nothing else changes.

What the compose version adds, and why

`docker run` with SQLite is fine for one person on a laptop. `docker-compose.yml` in this repo runs the same n8n image next to a Postgres container because that is what a company runs: Postgres can be backed up and shared by several n8n processes, SQLite cannot. Same workflows, same JSON, different database setting.